

Ch15: Comparing Sorting Methods

Slide 1: Three Sorting Methods

In the previous chapters, we learned three basic sorting algorithms:

- **Bubble Sort**
- **Selection Sort**
- **Insertion Sort**

All three can arrange an array in **ascending** or **descending** order.

The important point is that they all have the same overall goal:

Take an unsorted array and rearrange its elements into the required order.

For example:

Before sorting:

40 10 30 20

After sorting:

10 20 30 40

The final result is the same, but the **way each algorithm reaches that result is different**.

Think of three students asked to arrange the same stack of books by height:

- One repeatedly compares neighboring books.
- One searches for the shortest remaining book.
- One takes each book and inserts it into the correct position.

Same destination. Different strategy.

The three mental models

Bubble Sort

"Compare neighbors"

Selection Sort

"Find the smallest"

Insertion Sort

"Insert into the sorted part"

Key Idea

Do not judge sorting algorithms only by their final output.

All three can produce:

5 10 15 20 25

What matters is **how they move and organize the elements to get there.**

Slide 2: How the Three Methods Work

The easiest way to compare sorting algorithms is to identify their **main operation**.

Method	Main Idea	What it Mainly Does
Bubble Sort	Compare adjacent elements	Swaps neighboring elements
Selection Sort	Find minimum/maximum	Selects an element and places it
Insertion Sort	Insert into sorted portion	Shifts elements and inserts one value

1. Bubble Sort: Compare Neighbors

Bubble Sort looks at elements next to each other.

5 2 8 1
↑ ↑
compare

If they are in the wrong order:

5 > 2

they are swapped:

2 5 8 1

The algorithm continues comparing neighboring elements.

Its mental model is:

COMPARE → SWAP → MOVE FORWARD

2. Selection Sort: Find the Minimum

Selection Sort looks through the unsorted portion and finds the smallest element.

Example:

29 10 14 37 13
└──────────┘
unsorted part

Smallest value:

10

Place it at the beginning:

10 29 14 37 13

Then repeat for the remaining part.

Its mental model is:

FIND SMALLEST → PLACE IT → REPEAT

3. Insertion Sort: Insert Into Sorted Part

Insertion Sort treats part of the array as already sorted.

Example:

5 | 3 8 1

The 5 is currently the sorted portion.

Take 3:

5 3

Since 3 < 5, move 5 right and insert 3:

3 5 | 8 1

Its mental model is:

TAKE VALUE → SHIFT LARGER VALUES → INSERT

Key Idea

The three algorithms differ mainly in **what they focus on**:

Bubble → Neighbor

Selection → Minimum/Maximum

Insertion → Sorted portion

Slide 3: Beginner-Level Comparison

At the beginner level, it is useful to compare the algorithms by the operation you actually see in the code.

Feature	Bubble Sort	Selection Sort	Insertion Sort
Main operation	Swap neighbors	Select minimum/maximum	Shift and insert
Comparison style	Adjacent elements	Current minimum vs others	Key vs previous elements
Uses loops	Yes	Yes	Yes
Easy to understand	Yes	Yes	Yes
Swapping required	Frequently	Usually once per pass	Not necessarily
Builds sorted portion	From the end	From the beginning	From the beginning
Basic idea	Repeated neighboring comparisons	Repeated selection	Repeated insertion

Example: Same Array

Suppose we have:

5 3 8 1

Each method approaches it differently.

Bubble Sort

It begins with neighboring values:

5 3 8 1

↑ ↑

Swap because 5 > 3:

3 5 8 1

Then continue comparing neighbors.

Selection Sort

It looks for the smallest value in the unsorted portion:

5 3 8 1

find minimum

The minimum is:

1

Place it at the beginning:

1 3 8 5

The sorted portion is now:

1 |

The remaining values are still unsorted.

Insertion Sort

It considers part of the array sorted and inserts the next value into its correct position.

5 | 3 8 1

Take 3:

5 3

Shift 5:

_ 5

Insert 3:

3 5 | 8 1

Now the sorted portion has grown:

3 5 | 8 1

Important Observation

All three algorithms eventually produce the same sorted array.

The difference is **how they organize the work**.

Bubble Sort

neighbor → neighbor → neighbor

Selection Sort

search → select → place

Insertion Sort

take → shift → insert

Slide 4: When to Think of Each Method

A beginner does not need to memorize complicated descriptions. A strong mental shortcut is often more useful.

Bubble Sort

Think:

"Compare neighbors."

When you see Bubble Sort, imagine:

[5] [2]

↑ ↑

compare

If the values are in the wrong order:

[2] [5]

The algorithm repeatedly performs this kind of neighboring comparison.

Selection Sort

Think:

"Find the smallest."

Imagine looking through the remaining values and asking:

Which is the smallest?

For:

7 4 9 2 6

the smallest is:

2

Place it at the correct position:

2 4 9 7 6

Then search the remaining unsorted portion again.

The mental process is:

SEARCH → FIND MINIMUM → PLACE

Insertion Sort

Think:

"Insert into the sorted part."

Imagine holding cards in your hand.

You already have:

3 7 10

Now you receive:

5

You do not throw everything away and start over.

You find the correct position:

3 5 7 10

Insertion Sort works with the same basic idea.

The mental process is:

TAKE → SHIFT → INSERT

Quick Memory Table

Algorithm	Think
Bubble Sort	Compare neighbors
Selection Sort	Find the smallest
Insertion Sort	Insert into sorted part

Key Idea

If you can remember these three phrases, you already have the basic mental model:

Bubble → Compare neighbors

Selection → Find the smallest

Insertion → Insert into sorted part

Slide 5: Comparing the Methods as a Programmer

Now we can compare the three algorithms as programming techniques.

All three can be written using loops and conditions, but their **internal logic is different**.

Same Goal

Input:

40 10 30 20

↓ sorting

Output:

10 20 30 40

Different Strategy

Bubble Sort

Compare nearby values



Swap when necessary



Repeat

Selection Sort

Search remaining values



Find minimum



Place minimum



Repeat

Insertion Sort

Take next value



Compare with sorted portion



Shift larger values



Insert value



Repeat

Beginner Comparison

Question	Bubble	Selection	Insertion
What do I look at?	Neighbors	Unsorted portion	Sorted portion + next value

What do I mainly do?	Swap	Select	Shift and insert
Where does sorted data grow?	From the end	From the beginning	From the beginning
Main phrase	Compare neighbors	Find smallest	Insert correctly
Basic difficulty	Easy	Easy	Easy

A Useful Way to Identify the Algorithm

When reading C code, look for the main behavior:

Adjacent elements are compared



Bubble Sort

Minimum element is searched for



Selection Sort

Elements are shifted to make space

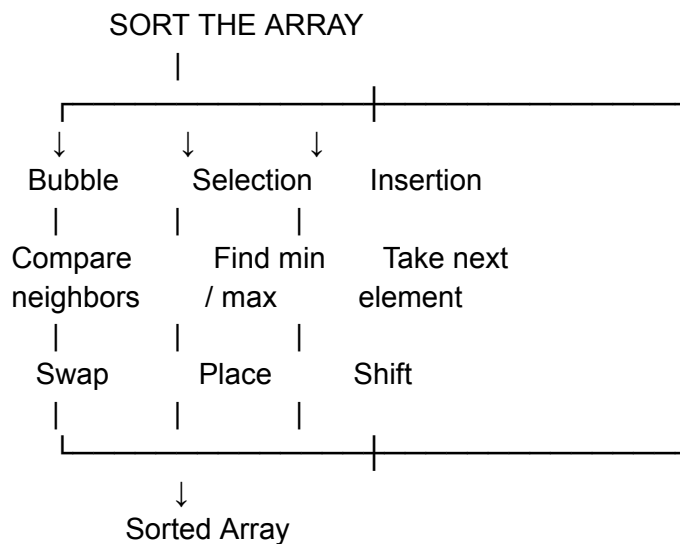


Insertion Sort

This is more useful than trying to memorize three blocks of code without understanding them.

Final Mental Model

Think of the three algorithms as three different ways of organizing the same messy array:



Core Takeaway

The three sorting methods have the same goal, but different strategies:

Bubble Sort compares neighboring elements.

Selection Sort finds an element to place.

Insertion Sort inserts each new element into the correct position.

Once these three ideas are clear, the actual C code becomes much easier to read because the code is no longer a mysterious collection of loops and swaps. It is simply the programming version of the strategy.